

Painel Terra

Documentação técnica

Como o backend e o frontend funcionam, por dentro, arquivo por arquivo — cronologia completa desde o momento em que você liga tudo até uma pergunta chegar no chat e voltar respondida. Fontes de dados, configuração do MCP, do agente e da autenticação, com destaque.

Sumário

1. Visão geral

2. Cronologia — como tudo se conecta

- Subindo o backend
- Subindo o frontend
- Primeira visita: a checagem de sessão
- Fazendo login
- Uma pergunta no chat, de ponta a ponta

3. Backend, arquivo por arquivo

- main.py
- auth.py
- agente.py
- criar_usuario.py
- Arquivos de configuração (.env, usuarios.yaml)

4. Frontend, arquivo por arquivo

5. Fontes de dados, em profundidade

6. Configuração do MCP, em profundidade

7. Configuração do agente, em profundidade

8. Autenticação, em profundidade

9. Onde mexer — índice rápido

10. Erros conhecidos e como diagnosticar

1. Visão geral

O Painel Terra são **dois programas separados** que conversam entre si — não um só. Isso é diferente da primeira tentativa (Streamlit), onde tudo vivia num processo único; a troca pra React + FastAPI trouxe apelo visual de verdade, mas também trouxe essa peça a mais pra entender: agora existem dois terminais abertos ao mesmo tempo, e os dois precisam estar rodando pro painel funcionar.

- **Frontend** — o que você vê no navegador. Código em `frontend/`, roda com `npm run dev`, sobe em `http://localhost:5173`. É só interface: HTML/CSS/JS. Não sabe nada de banco de dado, nem de IA — só sabe pedir coisas pro backend e desenhar o que volta.
- **Backend** — o cérebro. Código em `backend/`, roda com `uvicorn main:app`, sobe em `http://localhost:8001`. É quem confere login, quem conversa com o agente de IA, e quem aciona os MCP servers.

O frontend nunca fala com o backend num endereço diferente por acaso — o arquivo `vite.config.js` tem um *proxy* que redireciona toda chamada pra `/api/...` pro backend automaticamente. Você não precisa configurar CORS na mão nem lembrar a porta certa em cada chamada do frontend — só usar `/api/...` e o Vite cuida do resto, contanto que o backend esteja rodando na porta 8001 (se não estiver, é a causa mais comum de "não conecta em nada" — ver seção 10).

Por trás do backend, quatro **MCP servers** — os mesmos que você já tinha registrado no Claude Code antes de qualquer coisa disso existir: Zeus, Protector, o operacional (hoje nomeado `TESTE_INTEGRACAO`), e o `relatorios-terra` que gera PDF. O backend não reimplementa nada do que esses servidores fazem — ele só sabe *chamar* eles, exatamente como o Claude Code já sabia fazer.

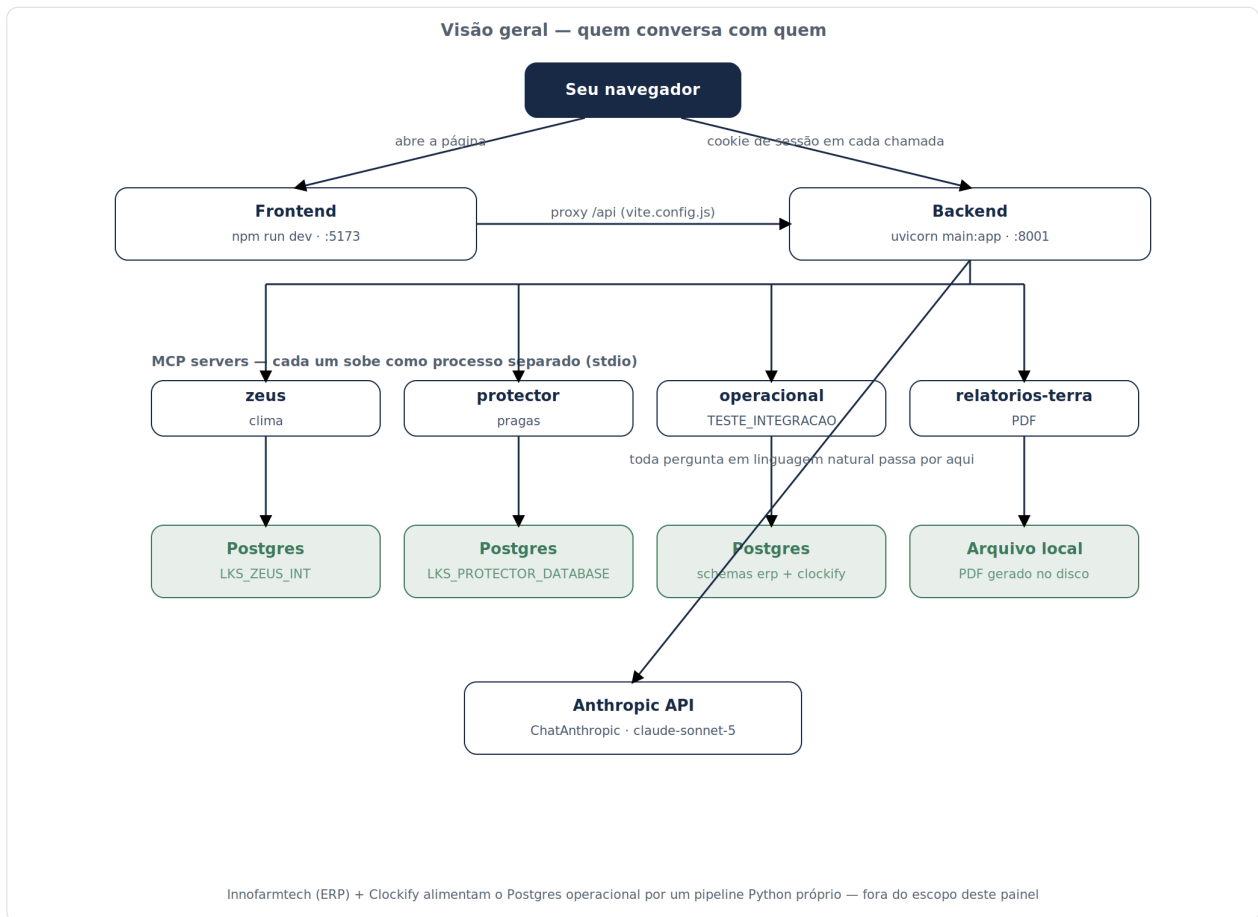


Figura 1 — quem conversa com quem, do navegador até o Postgres real.

Uma peça a mais que não aparece nesse diagrama por ser externa a tudo isso: a **Anthropic API**. É pra lá que vai toda pergunta em linguagem natural — o backend manda a pergunta (mais o histórico da conversa, mais a lista de ferramentas disponíveis) pro modelo Claude, e é o modelo quem decide, sozinho, se responde direto ou se pede pra chamar uma das ferramentas dos MCP servers. Isso é o "agente": não é um fluxo fixo escrito à mão, é o modelo decidindo em tempo real.

Por que dois processos, e não um só? Porque são duas linguagens e dois papéis diferentes. React (JavaScript) é o que sabe desenhar interface bonita; Python é o que sabe falar com LangGraph, MCP e Postgres. Dá pra fazer os dois num processo só (era o que o Streamlit fazia), mas aí perde o controle fino de HTML/CSS que motivou a troca de stack.

2. Cronologia — como tudo se conecta

Esta seção não é organizada por arquivo — é organizada pelo **tempo**, na ordem em que as coisas realmente acontecem quando você usa o painel. É a forma mais direta de entender o sistema inteiro: cinco momentos, do primeiro comando que você digita até uma resposta aparecer na tela.

- 2.1 — Você sobe o backend
- 2.2 — Você sobe o frontend
- 2.3 — Você abre o navegador pela primeira vez
- 2.4 — Você faz login
- 2.5 — Você manda uma pergunta no chat

2.1 — Subindo o backend

Tudo começa com um comando no terminal:

```
1 cd painel/backend
2 uvicorn main:app --reload --port 8001
```

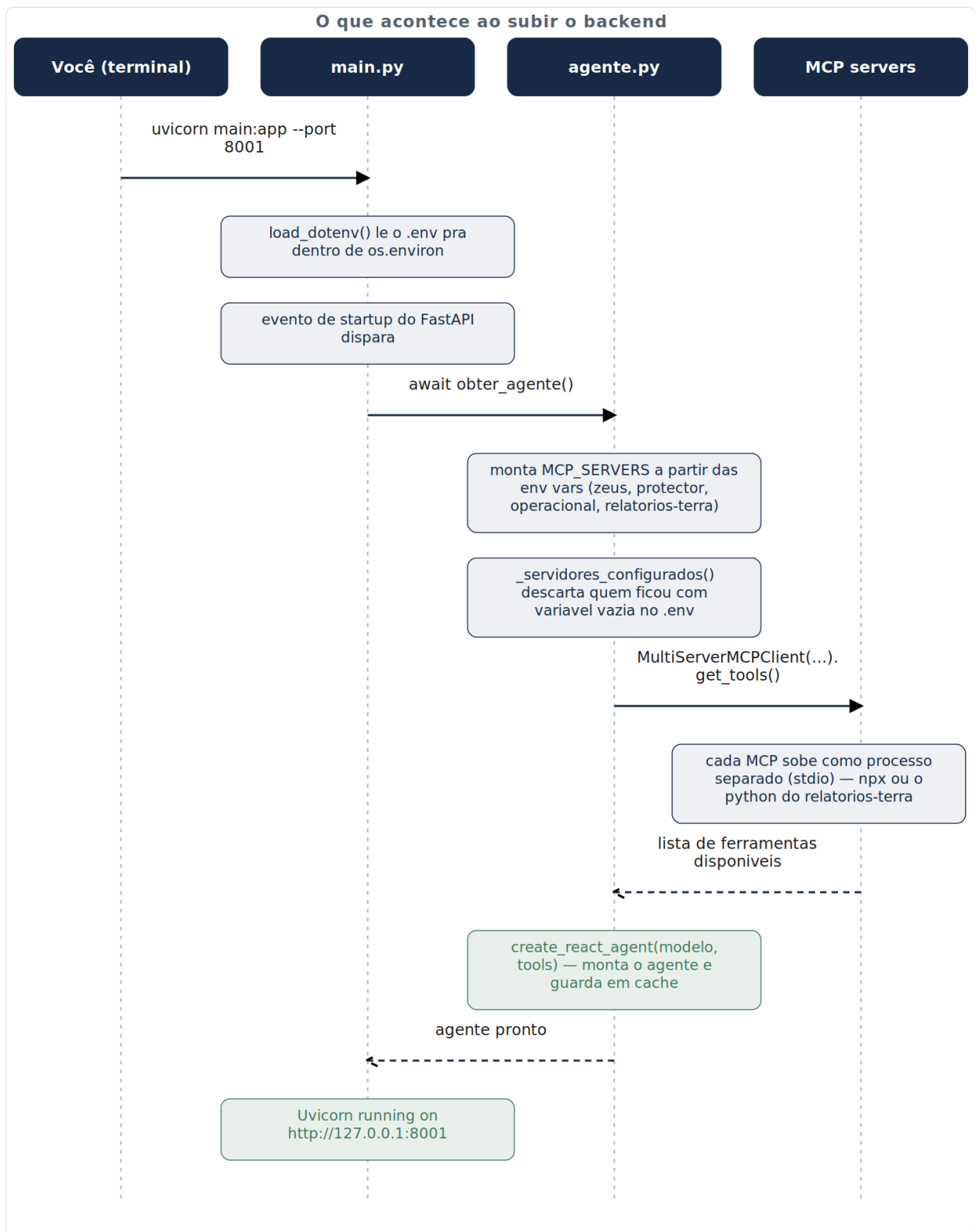


Figura 2 – sequência completa do startup do backend.

Passo a passo do que acontece por dentro:

1. `load_dotenv()` — a primeira linha executável de `main.py`. Lê o arquivo `.env` e coloca cada variável dentro de `os.environ`, onde o resto do código Python consegue ler. Se o

`.env` não existir ou estiver com nome errado (tem que ser exatamente `.env`, sem sobrenome), essa linha simplesmente não encontra nada pra carregar — **não dá erro**, só deixa as variáveis vazias, o que causa sintomas confusos mais adiante (ver seção 10).

2. **O evento de startup do FastAPI dispara** — a função `montar_agente_na_subida()` em `main.py`, marcada com `@app.on_event("startup")`, roda automaticamente assim que o Uvicorn termina de subir o servidor, antes de aceitar qualquer requisição.
3. Essa função chama `obter_agente()`, que mora em `agente.py` — é aqui que a parte pesada acontece.
4. `obter_agente()` monta o dicionário `MCP_SERVERS`, lendo as variáveis de ambiente pra cada um dos quatro servidores (zeus, protector, operacional, relatorios-terra).
5. `_servidores_configurados()` filtra essa lista: qualquer servidor cujas variáveis estejam vazias no `.env` é descartado, silenciosamente. Isso é proposital — permite rodar o painel só com `relatorios-terra` configurado, por exemplo, sem os outros três travarem o startup inteiro.
6. Pros servidores que sobraram, `MultiServerMCPClient(...).get_tools()` sobe cada um como um processo separado (é por isso que, se você olhar o Gerenciador de Tarefas do Windows enquanto o backend roda, vai ver processos `node` ou `python` extras, filhos do processo do Uvicorn) e pergunta pra cada um: "quais ferramentas você tem?". A resposta é a lista de tools que o agente vai poder usar.
7. `create_react_agent(modelo, tools)` — do LangGraph — junta o modelo (Claude) com essa lista de ferramentas e monta o agente de verdade.
8. O agente pronto fica **guardado em cache** (a variável `_cache` no topo de `agente.py`) — ele só é montado uma vez, na subida. Cada pergunta de chat reaproveita o mesmo agente, não remonta a conexão com os MCP servers do zero a cada mensagem.

Se algum MCP demorar muito pra responder `get_tools()` (ex.: `npx` baixando o pacote pela primeira vez), o startup inteiro do backend fica esperando — é normal a primeira subida demorar mais que as seguintes.

2.2 — Subindo o frontend

Em outro terminal (o backend precisa continuar rodando no primeiro):

```
1 cd painel/frontend
2 npm run dev
```

Aqui é bem mais simples que o backend — não tem nenhuma conexão externa pra esperar. O **Vite** (a ferramenta que serve o React durante o desenvolvimento) sobe, compila o `src/main.jsx` e tudo que ele importa, e passa a servir isso em `http://localhost:5173`. Não existe "build" nessa etapa — o Vite manda o código quase como está, e recompila na hora sempre que você salva um arquivo (é o que dá aquela atualização instantânea no navegador enquanto se edita).

O arquivo `index.html` é o ponto de entrada: ele tem uma `<div id="root">` vazia e um `<script type="module" src="/src/main.jsx">`. Tudo que aparece na tela é desenhado por JavaScript dentro dessa div — não tem HTML pronto de antemão, é tudo montado em tempo real pelo React.

2.3 — Primeira visita: a checagem de sessão

Você abre `http://localhost:5173` no navegador. Antes de decidir se mostra a tela de login ou o painel de verdade, o React precisa descobrir se você já está logado — e não confia em memória local nenhuma pra isso, sempre pergunta pro backend.

Isso acontece em `AuthContext.jsx`, assim que o app monta:

```
1  useEffect(() => {
2    api.quemSou()
3      .then(setUsuario)
4      .finally(() => setCarregando(false));
5  }, []);
```

`api.quemSou()` chama `GET /api/me`. Duas respostas possíveis:

- **Tem cookie de sessão válido** (você logou antes e ele ainda não expirou) → o backend devolve `{"usuario": ..., "nome": ...}`, o `AuthContext` guarda isso, e `App.jsx` te leva direto pro Chat.
- **Não tem cookie, ou expirou** → o backend devolve `401`, `quemSou()` devolve `null`, e `App.jsx` te manda pra tela de Login.

Enquanto essa checagem não termina, `carregando` fica `true` e nada é mostrado — é por isso que a tela pode ficar em branco por uma fração de segundo antes de decidir qual tela mostrar. É proposital: mostrar o Login por um instante e trocar pro painel (ou vice-versa) pareceria um "pisca" estranho.

2.4 — Fazendo login

Você digita usuário e senha na tela de `Login.jsx` e aperta Entrar. A partir daqui, cinco arquivos diferentes participam, nesta ordem:

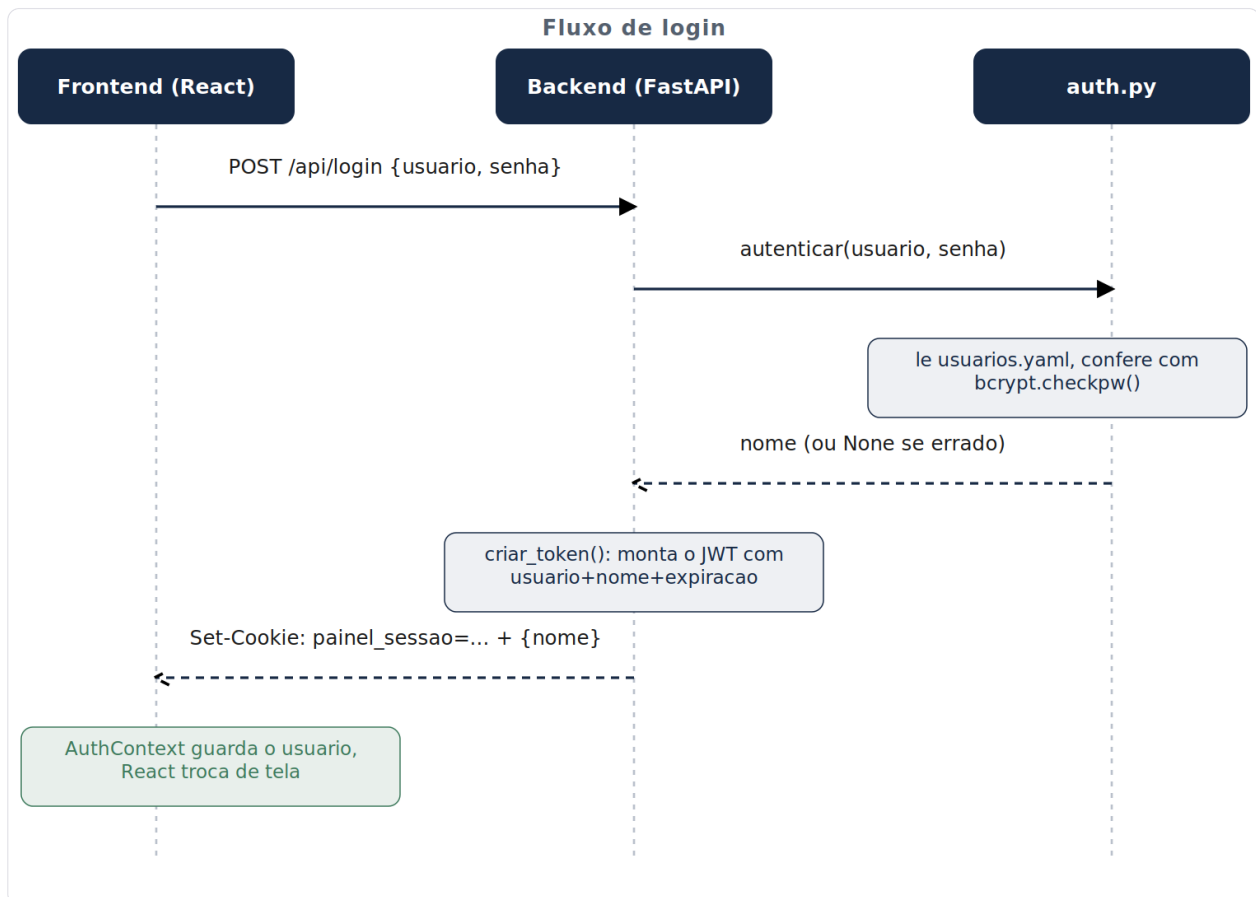


Figura 3 – sequência completa de um login.

1. `Login.jsx` chama `entrar(login, senha)`, do `AuthContext`.
2. `AuthContext` chama `api.login(...)`, que faz um `fetch` de `POST /api/login` com `{usuario, senha}` no corpo.
3. No backend, `main.py` recebe isso na rota `/api/login` e chama `autenticar(usuario, senha)`, de `auth.py`.
4. `autenticar()` lê `usuarios.yaml`, acha o usuário, e confere a senha com `bcrypt.checkpw()` — nunca compara texto puro com texto puro.
5. Se bateu, `main.py` chama `criar_token()`, que monta um **JWT** (explicado com calma na seção 8) contendo o usuário, o nome de exibição, e uma data de expiração 7 dias no futuro.
6. Esse token vai pro navegador dentro de um **cookie httpOnly** chamado `painel_sessao`, junto com `{"nome": "..."} no corpo da resposta.`

7. De volta no `AuthContext`, `setUsuario(dados)` atualiza o estado do React — e como `App.jsx` está observando esse estado, a troca de tela (de Login pro Chat) acontece sozinha, sem recarregar a página.

ONDE MEXER

Errou a senha três vezes e quer só testar rápido? O usuário de exemplo que vem no projeto é `admin` / `mudar123` — troque de verdade rodando `python criar_usuario.py` dentro de `backend/` (seção 3.4).

2.5 — Uma pergunta no chat, de ponta a ponta

Este é o fluxo mais importante do sistema inteiro — vale a pena ler com calma. É aqui que tudo que foi construído nesta conversa inteira (os MCP servers, o agente, a autenticação) se junta numa coisa só.

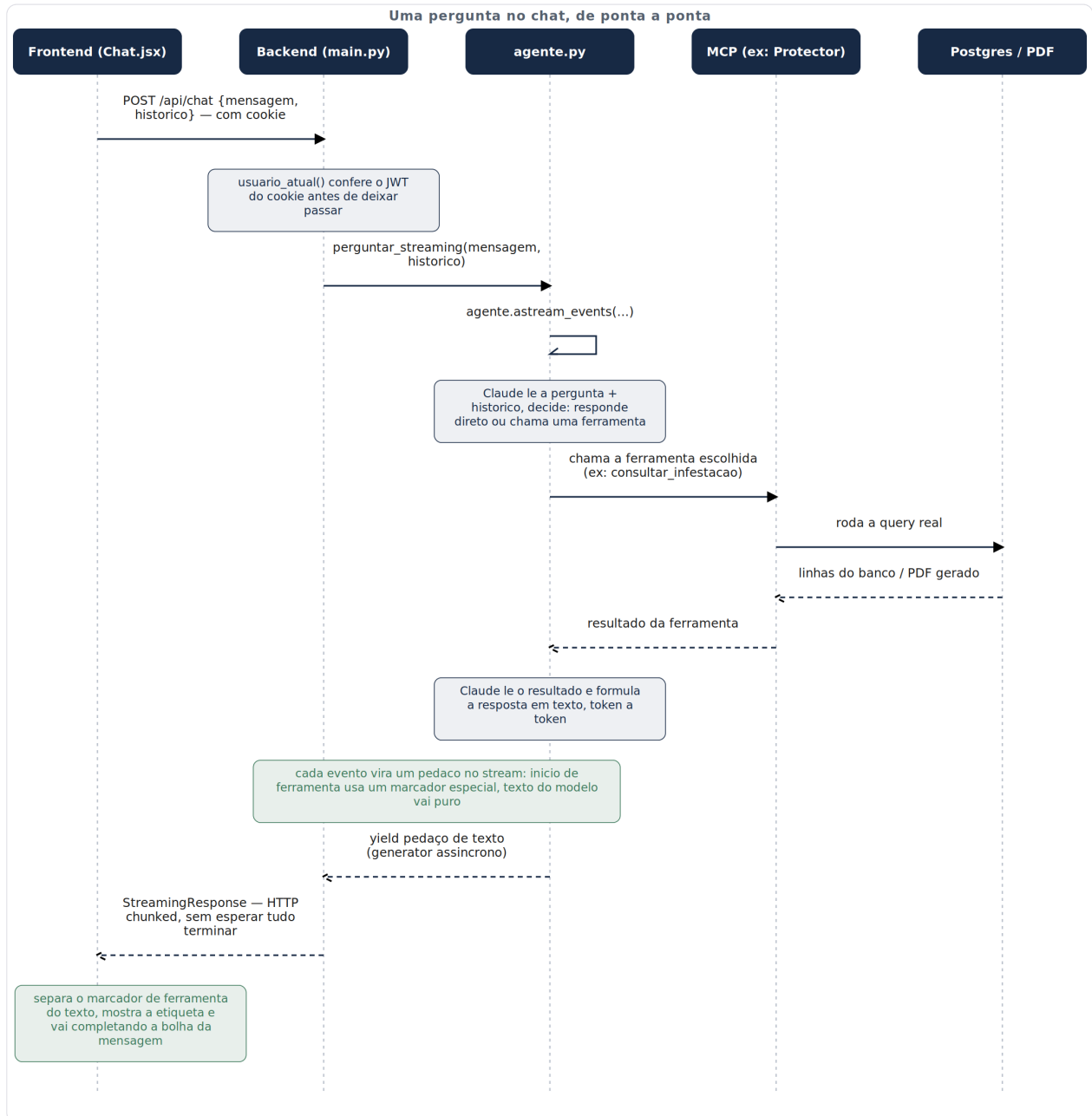


Figura 4 — uma pergunta no chat, do clique até o texto aparecer na tela.

A ida

1. Você digita a pergunta em `Chat.jsx` e aperta enviar (ou Enter).

2. `perguntarStreaming(...)`, em `api.js`, faz um `fetch` de `POST /api/chat` com a mensagem e o histórico da conversa — o cookie de sessão vai junto automaticamente (`credentials: "include"`).
3. No backend, antes de qualquer coisa, a rota confere quem está perguntando: `usuario_atual()` (de `auth.py`) lê o cookie, valida o JWT, e só deixa passar se a sessão for válida. Sem isso, qualquer pessoa na rede local poderia usar a API sem logar.
4. `main.py` chama `perguntar_streaming(...)`, de `agente.py` — e é aqui que o LangGraph entra em cena.

O raciocínio do agente

`agente.astream_events({"messages": mensagens}, version="v2")` manda a conversa inteira pro Claude, junto com a lista de ferramentas disponíveis (a mesma lista montada no startup, seção 2.1). O modelo lê a pergunta e decide, sozinho, uma de duas coisas:

- **Responder direto**, se souber a resposta sem precisar consultar nada (ex.: "o que é o Zeus?").
- **Chamar uma ferramenta**, se a pergunta precisar de dado real (ex.: "qual a pluviometria da fazenda Dourado essa semana?" → chama uma tool do MCP Zeus).

Quando decide chamar uma ferramenta, o LangGraph fala o protocolo MCP com o servidor certo (Zeus, Protector, Operacional ou relatorios-terra) — a mesma conversa que o Claude Code já sabia ter, só que agora automatizada dentro do backend. O MCP roda a query real no Postgres (ou gera o PDF de verdade, no caso do relatorios-terra) e devolve o resultado. O Claude lê esse resultado e formula a resposta final em texto — token a token, não tudo de uma vez.

A volta, em streaming

`astream_events` entrega tudo isso como uma sequência de eventos. `agente.py` escuta dois tipos:

- `on_tool_start` → o agente decidiu chamar uma ferramenta. Isso vira um marcador especial no texto (formato `FERRAMENTA:nome` entre dois bytes nulos) — é o que alimenta a etiqueta colorida ("Zeus · clima") que aparece acima da resposta no chat.
- `on_chat_model_stream` → um pedaço de texto de verdade, a resposta sendo formulada. Vira texto puro no stream.

Cada pedaço vira um `yield` dentro de um generator Python assíncrono — e o FastAPI manda cada `yield` pro navegador assim que ele existe, sem esperar a resposta inteira terminar (`StreamingResponse`, em `main.py`). Do lado do frontend, `perguntarStreaming()` lê esse fluxo pedaço a pedaço (`res.body.getReader()`) e `Chat.jsx` vai colando cada pedaço na bolha de mensagem — é isso que dá a sensação de "o texto indo aparecendo", igual ao Claude.ai.

Achado testando de verdade, não hipotético: se algo der errado no meio do stream (ex.: chave de API inválida), sem tratamento de erro a conexão simplesmente morre e o navegador não recebe nada — nem uma mensagem de erro. `perguntar_streaming()` tem um `try/except` em volta do laço inteiro exatamente por causa disso: qualquer exceção vira uma última mensagem de erro visível, em vez de sumir em silêncio.

3. Backend, arquivo por arquivo

Cinco arquivos de código, mais dois de configuração. Nesta ordem: o ponto de entrada (`main.py`), a autenticação, o agente, o script de criar usuário, e por fim os arquivos que você de fato edita no dia a dia (`.env` e `usuarios.yaml`).

3.1 — main.py

ARQUIVO	PAPEL	DEPENDE DE	CHAMADO POR
backend/main.py	ponto de entrada do backend, define as rotas HTTP	auth.py, agente.py	uvicorn (linha de comando)

É o arquivo que o comando `uvicorn main:app` carrega — `app` é o nome da variável que precisa existir nele (`app = FastAPI(...)`). Seis rotas, nesta ordem no arquivo:

Rota	O que faz	Precisa de login?
POST /api/login	Confere usuário/senha, devolve cookie de sessão	Não
POST /api/logout	Apaga o cookie	Não
GET /api/me	Devolve quem está logado (ou 401)	Sim
GET /api/status	Lista quais MCP servers entraram de verdade	Não*
POST /api/chat	A pergunta pro agente, em streaming	Sim

* `/api/status` hoje não exige login — é só a lista de nomes dos MCPs, sem dado sensível. Se isso mudar, considere proteger também.

As rotas marcadas "Sim" têm `usuario: dict = Depends(usuario_atual)` na assinatura — é assim que o FastAPI aplica a checagem de login: antes de rodar o corpo da função, ele roda `usuario_atual()` (de `auth.py`), e se essa função levantar um erro (sessão inválida), a rota inteira nunca executa.

```
1 @app.post("/api/chat")
2 async def chat(body: ChatBody, usuario: dict = Depends(usuario_atual)):
3     async def gerar():
4         async for pedaco in perguntar_streaming(body.mensagem, body.historico):
5             yield pedaco
6
7     return StreamingResponse(gerar(), media_type="text/plain")
```

Repare que a rota de chat não processa a pergunta ela mesma — ela só abre um `StreamingResponse` em volta do generator que vem de `perguntar_streaming()` (agente.py). `main.py` não sabe nada sobre LangGraph, MCP ou Postgres — só sabe roteamento HTTP e login. Essa separação é proposital: se um dia o jeito de gerar a resposta mudar, só `agente.py` muda.

CORS

`app.add_middleware(CORSMiddleware, ...)` permite que o frontend (rodando em `localhost:5173`) fale com o backend (`localhost:8001`) — são portas diferentes, e o navegador bloqueia isso por padrão a menos que o backend autorize explicitamente. O endereço autorizado vem de `ORIGEM_FRONTEND`, no `.env`.

ONDE MEXER

Vai publicar o frontend num endereço diferente de `localhost:5173` mais adiante (produção)? Troca `ORIGEM_FRONTEND` no `.env` do backend — não precisa editar `main.py`.

3.2 — auth.py

ARQUIVO	PAPEL	DEPENDÊNCIA	CHAMADO POR
backend/auth.py	confere senha, monta e valida sessão (JWT)	usuarios.yaml, JWT_SECRET (.env)	main.py

Três funções, cada uma faz uma coisa só — vale entender as três de verdade, é a seção 8 (Autenticação em profundidade) que aprofunda o "por quê"; aqui é só o "o quê".

autenticar(usuario, senha)

```
1 def autenticar(usuario: str, senha: str) -> str | None:
2     """Confere usuário/senha. Retorna o nome de exibição se certo, None se
   errado."""
3     usuarios = _carregar_usuarios()
4     dados = usuarios.get(usuario)
5     if not dados:
6         return None
7     if not bcrypt.checkpw(senha.encode(), dados["senha_hash"].encode()):
8         return None
9     return dados.get("nome", usuario)
```

Lê o `usuarios.yaml` inteiro toda vez que é chamada (não guarda em cache — arquivo pequeno, não compensa a complexidade de invalidar cache quando o arquivo muda). `bcrypt.checkpw()` é o único lugar do sistema inteiro que sabe se uma senha está certa — nem o `main.py` nem o frontend nunca veem a senha em texto puro depois desse ponto.

criar_token(usuario, nome)

```
1 def criar_token(usuario: str, nome: str) -> str:
2     expira = datetime.now(timezone.utc) + timedelta(days=SESSAO_DIAS)
3     return jwt.encode(
4         {"sub": usuario, "nome": nome, "exp": expira}, JWT_SECRET,
5         algorithm=JWT_ALGO
6     )
```

`jwt.encode(...)` não criptografa o conteúdo — **qualquer pessoa consegue ler** o usuário e o nome de dentro do token (é só texto codificado em base64, não cifrado). O que garante que ninguém *forja* um token válido é a assinatura, calculada com `JWT_SECRET` — sem saber esse segredo, dá pra ler o token mas não dá pra criar um novo que passe na validação.

usuario_atual(...)

```
1 def usuario_atual(painel_sessao: str | None = Cookie(default=None)) -> dict:
2     """Dependency do FastAPI – protege qualquer rota que precisar de login."""
3     if not JWT_SECRET:
4         raise HTTPException(500, "JWT_SECRET não configurado no .env do backend.")
5     if not painel_sessao:
6         raise HTTPException(401, "Não autenticado.")
7     try:
8         payload = jwt.decode(painel_sessao, JWT_SECRET, algorithms=[JWT_ALGO])
9     except JWTError:
10         raise HTTPException(401, "Sessão inválida ou expirada.")
11     return {"usuario": payload["sub"], "nome": payload["nome"]}
```

É uma *dependency* do FastAPI, não uma função chamada diretamente — o `Cookie(default=None)` na assinatura instrui o FastAPI a puxar o cookie `painel_sessao` da requisição sozinho, sem precisar disso escrito à mão em cada rota. As rotas protegidas só declaram `Depends(usuario_atual)` e ganham isso de graça.

Repare no `if not JWT_SECRET` : se o `.env` não tiver essa variável, toda rota protegida devolve erro 500 em vez de deixar todo mundo passar sem checar nada — falha fechada, não aberta. É proposital.

ONDE MEXER

Quer que a sessão dure mais ou menos que 7 dias? Troca `SESSAO_DIAS`, no topo do arquivo.

3.3 — agente.py

ARQUIVO	PAPEL	DEPENDÊNCIA	CHAMADO POR
backend/agente.py	monta o agente LangGraph, conecta nos MCP servers, produz a resposta em streaming	variáveis de ambiente dos 4 MCPs, ANTHROPIC_API_KEY	main.py

É o arquivo mais importante do backend — praticamente tudo que as seções 5, 6 e 7 deste documento aprofundam mora aqui dentro. Quatro partes: a lista de servidores (`MCP_SERVERS`), o filtro de quem está pronto (`_servidores_configurados()`), a montagem do agente (`obter_agente()`), e a produção da resposta (`perguntar_streaming()`).

MCP_SERVERS — o dicionário central

```
1 MCP_SERVERS = {
2     "zeus": {
3         "command": "npx",
4         "args": ["-y", "@modelcontextprotocol/server-postgres",
5                 os.environ.get("ZEUS_CONNECTION_STRING", "")],
6         "transport": "stdio",
7     },
8     "protector": {
9         "command": "npx",
10        "args": ["-y", "@modelcontextprotocol/server-postgres",
11                os.environ.get("PROTECTOR_CONNECTION_STRING", "")],
12        "transport": "stdio",
13    },
14    "operacional": {
15        # TESTE_INTEGRACAO no `claude mcp list`. Roda `claude mcp get
16        # TESTE_INTEGRACAO`
17        # e confirma comando/args reais — não é garantido ser o mesmo pacote npx do
18        # zeus/protector, já que essa base nasceu de um pipeline Python próprio.
19        "command": os.environ.get("OPERACIONAL_COMMAND", ""),
20        "args": os.environ.get("OPERACIONAL_ARGS", "").split("|") if
21        os.environ.get("OPERACIONAL_ARGS") else [],
22        "transport": "stdio",
23    },
24    "relatorios-terra": {
25        "command": os.environ.get("PYTHON312_PATH", r"C:\Program
26        Files\Python312\python.exe"),
27        "args": [os.environ.get("RELATORIOS_SERVER_PATH",
28                                r"C:\Workspace\Jarvs -
29        Terra\mcp_relatorios\server.py")],
30        "transport": "stdio",
31    },
32 }
```

```
27     },
28 }
```

Cada chave ("zeus", "protector", etc.) é um nome **interno** — é o que aparece na etiqueta colorida do chat (seção 2.5) e é o nome que o LangGraph usa pra se referir ao servidor. Ele **não precisa bater** com o nome que aparece no `claude mcp list` — são dois sistemas de nomenclatura diferentes que só coincidem por conveniência nos três primeiros. Cada entrada tem sempre três campos:

Campo	O que é	Exemplo real
<code>command</code>	o executável que sobe o servidor	<code>npx</code> ou o caminho do <code>python.exe</code>
<code>args</code>	lista de argumentos pra esse comando	<code>["-y", "@modelcontextprotocol/server-postgres", "postgresql://..."]</code>
<code>transport</code>	como conversar com o processo — sempre <code>"stdio"</code> aqui	—

Repare que **zeus** e **protector** usam o mesmo `command` (`npx`) e o mesmo pacote (`@modelcontextprotocol/server-postgres`) — só a connection string no final de `args` muda, e ela vem de uma variável de ambiente diferente para cada um. Já **operacional** não assume nada sobre qual comando usar — como esse MCP nasceu de um pipeline Python próprio (não do pacote genérico de Postgres), o comando inteiro vem do `.env`, sem valor padrão. E **relatorios-terra** é o único que não é um banco de dado — é o `mcp_relatorios/server.py` que vocês já tinham, rodando com o Python 3.12 específico da máquina.

_servidores_configurados() — o filtro

```
1 def _servidores_configurados() -> dict:
2     return {
3         nome: cfg for nome, cfg in MCP_SERVERS.items()
4         if cfg.get("command") and all(cfg.get("args", []))
5     }
```

Antes de tentar subir qualquer servidor, essa função descarta qualquer entrada cujo `command` esteja vazio ou cujos `args` tenham algum item vazio. É o que permite rodar o painel com só um ou dois MCPs configurados — por exemplo, testando só o `relatorios-terra` sem ainda ter preenchido Zeus/Protector/Operacional no `.env`. Sem esse filtro, o `MultiServerMCPClient` tentaria subir um processo com comando vazio e o backend inteiro falharia ao iniciar.

obter_agente() — a montagem, com cache

```
1 _cache = {"agente": None, "nomes": []}
2
3
4 async def obter_agente():
5     """Monta o agente uma vez só (chamado no startup do FastAPI) e reaproveita a
6     mesma instância – cada requisição de chat não remonta a conexão com os MCPs."""
7     if _cache["agente"] is not None:
8         return _cache["agente"], _cache["nomes"]
9
10    servidores = _servidores_configurados()
11    if not servidores:
12        return None, []
13
14    client = MultiServerMCPClient(servidores)
15    tools = await client.get_tools()
16    modelo = ChatAnthropic(model=MODELO)
17    agente = create_react_agent(modelo, tools)
18
19    _cache["agente"] = agente
20    _cache["nomes"] = list(servidores.keys())
21    return agente, _cache["nomes"]
```

`_cache` é um dicionário simples no nível do módulo — existe enquanto o processo do backend estiver de pé. Na **primeira** chamada (disparada pelo evento de startup, seção 2.1), ele de fato conecta nos MCPs e monta o agente; em **todas as chamadas seguintes** (uma por pergunta de chat), ele só devolve o que já estava guardado — não reconecta, não remonta nada. É por isso que a primeira pergunta depois de subir o backend não é mais lenta que as seguintes: o trabalho pesado já foi feito no startup, não na pergunta.

perguntar_streaming() — onde a resposta é produzida

```
1 async def perguntar_streaming(mensagem: str, historico: list):
2     """Gerador async que produz pedaços de texto conforme o modelo responde.
3     historico: lista de dicts {"role": "user"|"assistant", "content": "..."}."""
4     agente, nomes = await obter_agente()
5     if agente is None:
6         yield "Nenhum MCP server configurado ainda – confere o .env do backend."
7         return
8
9     mensagens = historico + [{"role": "user", "content": mensagem}]
10    try:
11        async for evento in agente.astream_events({"messages": mensagens},
12            version="v2"):
13            tipo = evento["event"]
14            if tipo == "on_tool_start":
15                # marcador que o front separa do texto normal – mostra qual base
```

```

15         # respondeu, não só a resposta. Ver PAINEL-STATUS.md / router.md:
16         # "deixe claro de onde veio a informação" já era princípio do
projeto.
17         yield f"\x00FERRAMENTA:{evento['name']}\x00"
18     elif tipo == "on_chat_model_stream":
19         chunk = evento["data"]["chunk"]
20         texto = getattr(chunk, "content", "")
21         if isinstance(texto, list):
22             texto = "".join(
23                 p.get("text", "") for p in texto if isinstance(p, dict)
24             )
25         if texto:
26             yield texto
27 except Exception as e:
28     # Sem isso, um erro no meio do stream (ex.: chave de API inválida) só
29     # derruba a conexão sem nenhuma mensagem – achado testando de verdade,
30     # não hipotético. Client precisa ver ALGO em vez de resposta vazia.
31     yield f"\n\n[erro consultando o agente: {e}]"

```

`agente.astream_events(...)` é a chamada que efetivamente manda a pergunta pro Claude. Ela devolve um fluxo de *eventos* — não só texto — e esta função filtra dois tipos específicos, ignorando todo o resto (existem outros tipos de evento no protocolo, como início/fim de cada etapa do grafo, que não importam pra o que o chat precisa mostrar):

- `"on_tool_start"` — vira o marcador de ferramenta (linha 95)
- `"on_chat_model_stream"` — vira texto puro (linhas 96-104), com um cuidado a mais: o conteúdo do `"chunk"` às vezes vem como lista de blocos em vez de string direta (depende de como o modelo formatou aquele pedaço específico), e o `isinstance(texto, list)` trata os dois casos.

O `except Exception as e` no final (linha 105) é deliberadamente amplo — pega qualquer erro, não só os esperados. Isso é consciente: numa função de streaming, um erro não tratado no meio do laço mata a conexão em silêncio (foi exatamente o que aconteceu no teste com uma chave de API inválida, antes dessa correção). Melhor mostrar o erro cru pro usuário do que ele ver uma resposta vazia sem entender por quê.

ONDE MEXER

Quer adicionar um **quinto** MCP server no futuro? Copia o bloco de uma das entradas existentes dentro de `MCP_SERVERS`, troca o nome da chave e os valores, e adiciona as variáveis de ambiente correspondentes no `.env.example` e no `.env` real. Não precisa mexer em mais nada — nem em `_servidores_configurados()`, nem em `main.py`, nem no frontend.

3.4 — criar_usuario.py

ARQUIVO	PAPEL	DEPENDENTE DE	CHAMADO POR
backend/ criar_usuario.py	ferramenta de linha de comando pra adicionar/trocar usuário	bcrypt, yaml	você, na mão, no terminal

Não faz parte do backend rodando — é um script separado, que você roda uma vez por usuário que precisa criar ou atualizar:

```
1 cd backend
2 python criar_usuario.py
```

```
1 def main():
2     if not CONFIG_PATH.exists():
3         dados = {"usuarios": {}}
4     else:
5         with open(CONFIG_PATH, "r", encoding="utf-8") as f:
6             dados = yaml.safe_load(f) or {"usuarios": {}}
7
8     login = input("Login (sem espaço, ex: joao): ").strip()
9     nome = input("Nome de exibição: ").strip()
10    senha = getpass.getpass("Senha: ")
11
12    dados.setdefault("usuarios", {})[login] = {
13        "nome": nome,
14        "senha_hash": bcrypt.hashpw(senha.encode(), bcrypt.gensalt()).decode(),
15    }
16
17    with open(CONFIG_PATH, "w", encoding="utf-8") as f:
18        yaml.safe_dump(dados, f, allow_unicode=True)
19
20    print(f"Usuário '{login}' salvo em {CONFIG_PATH}.")
```

Pergunta login, nome de exibição e senha (a senha não aparece na tela enquanto você digita — `getpass.getpass()` cuida disso), calcula o hash bcrypt de verdade, e grava (ou atualiza, se o login já existir) dentro de `usuarios.yaml`. Existe só por causa de uma regra simples: **nunca editar o hash de senha na mão**. Copiar/colar ou digitar um hash bcrypt errado é fácil e o erro só aparece depois, na hora do login, como "senha incorreta" mesmo com a senha certa — esse script elimina essa categoria de erro inteira.

Rodar de novo com um login que já existe **substitui** a senha antiga — não existe confirmação nem "tem certeza?" no meio do caminho. Isso também serve pra trocar de senha, não só pra criar usuário novo.

3.5 — Arquivos de configuração (.env e usuarios.yaml)

Estes dois são os únicos arquivos do backend que você deveria precisar editar no dia a dia — todo o resto é código que já funciona, e mexer nele é exceção, não regra. Os dois seguem a mesma disciplina do resto do projeto: existe uma versão `.example` segura pra copiar (sem segredo nenhum dentro, pode ficar no controle de versão) e uma versão real (nunca commitada — os dois têm entrada no `.gitignore`).

`.env.example` → `.env`

```
1 # Copia pra .env (sem o .example). .env NUNCA é commitado.
2
3 ANTHROPIC_API_KEY=
4
5 # Gera um valor novo com: python -c "import secrets; print(secrets.token_hex(32))"
6 # Não reaproveita o exemplo abaixo em produção — é só ilustrativo.
7 JWT_SECRET=af1a9c1404e02cc857cb156e7a693f73c0efce8f41b8d22a010b64ca0641d2d6
8
9 # Precisa bater com onde o `npm run dev` do frontend sobe (padrão do Vite: 5173)
10 ORIGEM_FRONTEND=http://localhost:5173
11
12 # --- Zeus e Protector
13 # -----
14 # Roda `claude mcp list` de dentro da pasta do projeto primeiro pra confirmar os
15 # NOMES
16 # reais visíveis aqui — vazamento de escopo pendente de resolver
17 # (LKS_PROTECTOR_DATABASE
18 # e relatorios-raizel aparecem mesmo de dentro da pasta Terra), então confere antes
19 # de
20 # assumir que são os nomes certos.
21 ZEUS_CONNECTION_STRING=postgresql://usuario:senha@host:5432/banco
22 PROTECTOR_CONNECTION_STRING=postgresql://usuario:senha@host:5432/banco
23
24 # --- Operacional (TESTE_INTEGRACAO: erp + clockify)
25 # -----
26 # `claude mcp get TESTE_INTEGRACAO` mostra comando e args reais. Se for o mesmo
27 # padrão
28 # do zeus/protector:
29 #   OPERACIONAL_COMMAND=npx
30 #   OPERACIONAL_ARGS=-y|@modelcontextprotocol/server-postgres|postgresql://
31 #   usuario:senha@host:5432/banco
32 # (args separados por | , não por vírgula — connection string não deve ter |
33 # dentro)
34 OPERACIONAL_COMMAND=
35 OPERACIONAL_ARGS=
36
37 # --- Relatórios (já confirmado no claude mcp list)
38 # -----
```

```
30 PYTHON312_PATH=C:\Program Files\Python312\python.exe
31 RELATORIOS_SERVER_PATH=C:\Workspace\Jarvs - Terra\mcp_relatorios\server.py
32
33 PAINEL_MODELO=claude-sonnet-5
```

Variável	Pra que serve	Onde é usada
ANTHROPIC_API_KEY	autentica as chamadas pro Claude	ChatAnthropic , em agente.py — lida automaticamente pela lib, não aparece explícita no código
JWT_SECRET	assina e valida os tokens de sessão	auth.py
ORIGEM_FRONTEND	quem tem permissão de chamar a API (CORS)	main.py
ZEUS_CONNECTION_STRING	connection string do Postgres do Zeus	agente.py, dentro de MCP_SERVERS
PROTECTOR_CONNECTION_STRING	idem, Protector	agente.py
OPERACIONAL_COMMAND / _ARGS	comando completo do MCP operacional (TESTE_INTEGRACAO)	agente.py
PYTHON312_PATH	caminho do Python 3.12 que roda o relatorios-terra	agente.py
RELATORIOS_SERVER_PATH	caminho do server.py do relatorios-terra	agente.py
PAINEL_MODELO	qual modelo Claude o agente usa	agente.py

usuarios.example.yaml → usuarios.yaml

```
1 # Copia pra usuarios.yaml (sem o .example). O hash abaixo é real, gerado com bcrypt
2 # de verdade – senha de exemplo é "mudar123". Depois de copiar, roda
3 # `python criar_usuario.py` pra trocar por usuários e senhas reais – não edita o
  hash
4 # na mão.
5 usuarios:
6   admin:
7     nome: Administrador
```

```
8      senha_hash: "$2b$12$0UqEjkCUr4fFU3pvkph1T.7k.GVn2e55WbVKD5B03/RNQN7bhc9PW"
```

Formato simples: uma chave `usuarios`, dentro dela um bloco por login, cada um com `nome` (o que aparece na sidebar do painel depois de logar) e `senha_hash` (nunca a senha em si). O exemplo já vem com um hash bcrypt real e funcional — login `admin`, senha `mudar123` — pra você conseguir testar o painel sem precisar rodar `criar_usuario.py` primeiro. Mas troque antes de usar de verdade.

ONDE MEXER

Nunca edite `senha_hash` na mão — sempre pelo `criar_usuario.py` (seção 3.4). Editar o `nome` direto no YAML é seguro, não precisa do script pra isso.

4. Frontend, arquivo por arquivo

Dez arquivos — bem mais curtos individualmente que o backend, porque cada um faz uma coisa só. A ordem aqui segue o caminho que uma requisição percorre: primeiro o que monta a aplicação, depois autenticação, depois comunicação com a API, depois as telas em si.

4.1 — main.jsx e index.html

ARQUIVO	PAPEL	DEPENDE DE	CHAMADO POR
frontend/src/ main.jsx	ponto de entrada — a primeira linha de JS que roda	App.jsx, index.css	index.html

```
1 import { StrictMode } from 'react'
2 import { createRoot } from 'react-dom/client'
3 import './index.css'
4 import App from './App.jsx'
5
6 createRoot(document.getElementById('root')).render(
7   <StrictMode>
8     <App />
9   </StrictMode>,
10 )
```

Curto de propósito: só encontra a `<div id="root">` (que vem de `index.html`) e manda o React desenhar `<App />` dentro dela. `<StrictMode>` é uma ferramenta de desenvolvimento do React que ajuda a pegar erro cedo (ela roda alguns efeitos duas vezes de propósito, em desenvolvimento, pra expor bugs de estado) — não afeta o funcionamento em produção.

ONDE MEXER

Título da aba do navegador e o ícone (favicon) ficam em `index.html`, na raiz de `frontend/` — não aqui.

4.2 — App.jsx

ARQUIVO	PAPEL	DEPENDE DE	CHAMADO POR
frontend/src/ App.jsx	define as rotas (URLs) da aplicação e protege as que exigem login	AuthContext, Layout, todas as páginas	main.jsx

Duas responsabilidades: decidir qual componente aparece pra cada URL, e impedir que quem não está logado veja qualquer coisa além da tela de login.

```
1 function Protegido({ children }) {
2   const { usuario, carregando } = useAuth();
3   if (carregando) return null;
4   if (!usuario) return <Navigate to="/login" replace />;
5   return children;
6 }
7
8 function RotaLogin() {
9   const { usuario, carregando } = useAuth();
10  if (carregando) return null;
11  if (usuario) return <Navigate to="/" replace />;
12  return <Login />;
13 }
```

`Protegido` envolve tudo que exige login — se `usuario` for `null` (não logado), manda pra `/login` em vez de mostrar o conteúdo. `RotaLogin` faz o oposto: se você já está logado e tenta acessar `/login` na mão pela URL, manda de volta pro Chat — não faz sentido ver a tela de login de novo.

As rotas propriamente ditas:

URL	Componente	Arquivo
<code>/login</code>	Login	pages/Login.jsx
<code>/</code>	Chat	pages/Chat.jsx
<code>/geral</code>	Placeholder	pages/Placeholder.jsx
<code>/clima</code>	Placeholder	pages/Placeholder.jsx
<code>/pragas</code>	Placeholder	pages/Placeholder.jsx
<code>/operacional</code>	Placeholder	pages/Placeholder.jsx

As quatro últimas usam o **mesmo componente** `Placeholder`, só com `titulo` e `descricao` diferentes — é assim que uma página vira "de verdade" na Fase 2: troca `<Placeholder .../>` pelo componente novo, sem tocar em mais nada nesse arquivo além dessa troca.

ONDE MEXER

Fase 2, criando a página de Clima de verdade? É aqui — troca a linha `<Placeholder titulo="Clima" .../>` por `<ClimaPage />` (o componente novo que você vai criar em `pages/Clima.jsx`), e adiciona o `import` no topo do arquivo.

4.3 — context/AuthContext.jsx

ARQUIVO	PAPEL	DEPENDÊNCIA	CHAMADO POR
frontend/src/ context/ AuthContext.jsx	guarda quem está logado, em memória, acessível de qualquer componente	lib/api.js	App.jsx (provedor), Layout.jsx e páginas (consumidores)

```
1 import { createContext, useContext, useEffect, useState } from "react";
2 import * as api from "../lib/api";
3
4 const AuthContext = createContext(null);
5
6 export function AuthProvider({ children }) {
7   const [usuario, setUsuario] = useState(undefined); // undefined = ainda checando
8   const [carregando, setCarregando] = useState(true);
9
10  useEffect(() => {
11    api.quemSou()
12      .then(setUsuario)
13      .finally(() => setCarregando(false));
14  }, []);
15
16  async function entrar(login, senha) {
17    const dados = await api.login(login, senha);
18    setUsuario(dados);
19  }
20
21  async function sair() {
22    await api.logout();
23    setUsuario(null);
24  }
25
26  return (
27    <AuthContext.Provider value={{ usuario, carregando, entrar, sair }}>
28      {children}
29    </AuthContext.Provider>
30  );
31 }
32
33 export function useAuth() {
34   return useContext(AuthContext);
35 }
```

É o padrão **Context** do React — em vez de passar `usuario` manualmente de componente em componente (o que ficaria repetitivo e frágil), qualquer componente da árvore chama

`useAuth()` e recebe `{ usuario, carregando, entrar, sair }` direto. `Layout.jsx` usa pra mostrar o nome e o botão de sair; `App.jsx` usa pra decidir qual tela mostrar; `Login.jsx` usa `entrar()` pra efetivamente logar.

Repare que `usuario` começa como `undefined` (diferente de `null`) — é assim que o código distingue "ainda não sei" de "sei que não está logado". `carregando` é o que `App.jsx` usa pra não tomar nenhuma decisão de rota antes da checagem de sessão (seção 2.3) terminar.

4.4 — lib/api.js

ARQUIVO	PAPEL	DEPENDE DE	CHAMADO POR
frontend/src/lib/api.js	toda comunicação HTTP com o backend passa por aqui, em um lugar só	nada (usa fetch nativo do navegador)	AuthContext.jsx, Chat.jsx

Cinco funções, uma por rota do backend (seção 3.1). Nenhum componente de tela faz `fetch()` diretamente — sempre chamam uma função daqui. Isso significa que se um dia o endereço da API mudar, ou se for preciso adicionar um cabeçalho em toda chamada, muda num lugar só.

```
1  /** Chama o chat em streaming, chamando onPedaco(texto) a cada pedaço recebido. */
2  export async function perguntarStreaming(mensagem, historico, onPedaco) {
3    const res = await fetch(`${BASE}/chat`, {
4      method: "POST",
5      headers: { "Content-Type": "application/json" },
6      credentials: "include",
7      body: JSON.stringify({ mensagem, historico }),
8    });
9    if (!res.ok || !res.body) {
10     const corpo = await res.json().catch(() => ({}));
11     throw new Error(corpo.detail || `Erro ${res.status}`);
12   }
13   const leitor = res.body.getReader();
14   const decodificador = new TextDecoder();
15   while (true) {
16     const { done, value } = await leitor.read();
17     if (done) break;
18     onPedaco(decodificador.decode(value, { stream: true }));
19   }
20 }
```

`perguntarStreaming` é a mais complexa — as outras quatro fazem um `fetch` normal e devolvem o JSON pronto; esta lê a resposta aos poucos. `res.body.getReader()` dá acesso ao fluxo de bytes brutos conforme eles chegam da rede, e o laço `while (true)` vai lendo pedaço a pedaço até `done` ficar verdadeiro. Cada pedaço decodificado vira uma chamada de `onPedaco(texto)` — é essa função de callback que `Chat.jsx` usa pra ir atualizando a tela.

Todo `fetch` tem `credentials: "include"` — sem isso, o navegador **não manda o cookie de sessão** nas chamadas, e toda rota protegida devolveria 401 mesmo com um

login válido. É um detalhe fácil de esquecer e que quebra a autenticação inteira silenciosamente se for removido.

`BASE = "/api"` — um caminho relativo, não `http://localhost:8001/api`. É o proxy do `vite.config.js` (seção 4.10) que traduz isso pro endereço real do backend. Enquanto os dois rodam nas portas padrão, isso é transparente.

4.5 — components/Layout.jsx

ARQUIVO	PAPEL	DEPENDE DE	CHAMADO POR
frontend/src/components/Layout.jsx	a casca visual — sidebar + área de conteúdo — que envolve todas as páginas logadas	AuthContext, react-router-dom, lucide-react (ícones)	App.jsx

É o único lugar do frontend com a sidebar de navegação — as cinco páginas (Chat, Geral, Clima, Pragas, Operacional) não desenham a própria barra lateral, elas só preenchem o espaço à direita dela. Isso acontece através de `<Outlet />`, do react-router-dom: é um "buraco" onde a rota ativa é desenhada.

```
1 const ITENS = [  
2   { to: "/", label: "Chat", icon: MessageCircle, fim: true },  
3   { to: "/geral", label: "Geral", icon: LayoutGrid },  
4   { to: "/clima", label: "Clima", icon: CloudRain },  
5   { to: "/pragas", label: "Pragas", icon: Bug },  
6   { to: "/operacional", label: "Operacional", icon: Users2 },  
7 ];
```

A lista de itens do menu é só isso — um array. Adicionar uma página nova ao menu é adicionar uma linha aqui (mais a rota correspondente em `App.jsx`, seção 4.2).

ONDE MEXER

Novo item no menu lateral? Adiciona um objeto nesse array `ITENS` — `to` (a URL), `label` (o texto), `icon` (importa de `lucide-react`, mesmo pacote dos que já estão em uso). Não esquece de criar a rota em `App.jsx` também.

4.6 — pages/Login.jsx

ARQUIVO	PAPEL	DEPENDE DE	CHAMADO POR
frontend/src/ pages/Login.jsx	a tela de login — formulário + o painel de marca à esquerda	AuthContext (função entrar), assets/ logo.png, assets/ marca-dagua.png	App.jsx

Dividida em duas metades (em telas largas — no celular, só o formulário aparece, a metade de marca fica escondida): à esquerda, o momento de identidade visual, com o fundo navy da marca e a marca d'água real (a mesma extraída e usada nos relatórios PDF, ver `mcp_relatorios/docs/marca-terra.md`); à direita, o formulário em si.

```
1  async function aoEnviar(e) {
2    e.preventDefault();
3    setErro("");
4    setEnviando(true);
5    try {
6      await entrar(login, senha);
7    } catch (e) {
8      setErro(e.message || "Não foi possível entrar.");
9    } finally {
10     setEnviando(false);
11   }
12 }
```

`aoEnviar` é a função que roda quando o formulário é submetido — chama `entrar(login, senha)` do `AuthContext` (seção 4.3), que por sua vez chama `api.login()` (seção 4.4). Se `entrar()` lançar um erro (senha errada, backend fora do ar, etc.), o `catch` guarda a mensagem em `erro` e o JSX mostra ela num aviso vermelho logo abaixo dos campos.

O botão fica desabilitado (`disabled={enviando || !login || !senha}`) enquanto os campos estão vazios ou enquanto uma tentativa de login já está em andamento — evita duplo-clique mandando duas tentativas de login ao mesmo tempo.

4.7 — pages/Chat.jsx

ARQUIVO	PAPEL	DEPENDE DE	CHAMADO POR
frontend/src/ pages/Chat.jsx	a tela de chat — a única página com dado real por trás nesta fase	lib/api.js (perguntarStreaming, status)	App.jsx, via Layout

O arquivo mais denso do frontend — vale entender em três partes: o estado que a tela guarda, a função que envia uma pergunta, e como a resposta em streaming é processada.

Estado

```
1 export default function Chat() {
2   const [mensagens, setMensagens] = useState([]);
3   const [entrada, setEntrada] = useState("");
4   const [enviando, setEnviando] = useState(false);
5   const [ativos, setAtivos] = useState([]);
6   const fimRef = useRef(null);
7
8   useEffect(() => {
9     status().then((s) => setAtivos(s.servidores_ativos || []));
10  }, []);
```

`mensagens` é a lista inteira da conversa (cada item tem `role`, `content`, e — só nas respostas do agente — `fontes`, a lista de MCPs que responderam). `ativos` vem de `GET /api/status` assim que a página monta, só pra mostrar "3 bases conectadas" no topo — não influencia o funcionamento do chat, é só informativo. `fimRef` + o segundo `useEffect` são o que faz a tela rolar sozinha pra última mensagem sempre que uma nova chega.

Enviando uma pergunta

```
1  async function enviar(e) {
2    e.preventDefault();
3    const texto = entrada.trim();
4    if (!texto || enviando) return;
5
6    const historico = mensagens.map(({ role, content }) => ({ role, content }));
7    setMensagens((m) => [...m, { role: "user", content: texto }]);
8    setEntrada("");
9    setEnviando(true);
10   setMensagens((m) => [...m, { role: "assistant", content: "", fontes: [] }]);
```

Repare na ordem: primeiro adiciona a mensagem do **usuário** na lista, depois já adiciona uma mensagem **vazia** do assistente (`content: ""`) — é essa mensagem vazia que vai sendo preenchida conforme o texto chega. É por isso que aparece uma bolha de resposta imediatamente, mesmo antes do primeiro pedaço de texto ter chegado (mostrando "Pensando..." — ver componente `Mensagem` mais abaixo).

Processando o streaming — a parte mais sutil do arquivo inteiro

```
1    await perguntarStreaming(texto, historico, (pedaco) => {
2      setMensagens((atual) => {
3        const copia = [...atual];
4        const ultima = { ...copia[copia.length - 1] };
5        let resto = pedaco;
6        // separa marcadores \x00FERRAMENTA:nome\x00 do texto normal
7        const re = /\x00FERRAMENTA:([^\x00]+)\x00/g;
8        let m;
9        let textoLimpo = "";
10       let cursor = 0;
11       while ((m = re.exec(resto))) {
12         textoLimpo += resto.slice(cursor, m.index);
13         if (!ultima.fontes.includes(m[1])) ultima.fontes = [...ultima.fontes,
14           m[1]];
15         cursor = re.lastIndex;
16       }
17       textoLimpo += resto.slice(cursor);
18       ultima.content += textoLimpo;
19       copia[copia.length - 1] = ultima;
20       return copia;
21     });
22   });
```

Cada vez que um pedaço de texto chega (lembrando: pode ser texto normal, ou pode conter o marcador de ferramenta que o backend insere — seção 2.5 e 3.3), este código:

1. Pega a **última** mensagem da lista (a do assistente, que está sendo construída aos poucos)
2. Usa uma expressão regular pra achar qualquer marcador de ferramenta escondido no pedaço recebido
3. Pra cada marcador achado, adiciona o nome da ferramenta em `fontes` (sem duplicar, por isso o `includes()` antes de adicionar) — é isso que vira a etiqueta colorida
4. Remove o marcador do texto (ele não deve aparecer pro usuário, é só sinalização interna) e concatena o restante em `content`

Tudo isso dentro de `setMensagens(atual => { ... })` — a forma "funcional" de atualizar estado no React, necessária aqui porque os pedaços chegam rápido e em sequência; usar

o valor de `mensagens` direto (em vez da função) arriscaria perder pedaços se dois chegassem muito próximos no tempo.

O componente Mensagem

```
1 function Mensagem({ role, content, fontes = [], erro }) {
2   if (role === "user") {
3     return (
4       <div className="flex justify-end">
5         <div className="max-w-[80%] rounded-lg rounded-tr-sm bg-terra-navy px-4
py-2.5 text-[15px] text-white">
6           {content}
7         </div>
8       </div>
9     );
10  }
```

Mensagens do usuário são sempre alinhadas à direita, em bolha navy sólida. Mensagens do assistente (o `return` logo depois, não mostrado aqui por brevidade) não usam bolha — só o texto direto, com a fileira de etiquetas de fonte acima quando existirem, e `"Pensando..."` em cinza claro enquanto `content` ainda está vazio.

ONDE MEXER

Quer mudar o texto de exemplo que aparece antes da primeira pergunta (o "Pergunta alguma coisa")? É o componente `EstadoVazio`, no final do arquivo. Quer mudar como o nome de cada MCP aparece na etiqueta ("Zeus · clima")? É o objeto `NOMES_LEGIVEIS`, no topo — a chave tem que bater exatamente com o nome usado em `MCP_SERVERS`, em `agente.py` (seção 3.3).

4.8 — pages/Placeholder.jsx

ARQUIVO	PAPEL	DEPENDE DE	CHAMADO POR
frontend/src/ pages/ Placeholder.jsx	tela de espera reaproveitada pelas 4 páginas da Fase 2	lucide-react	App.jsx, 4 vezes

```
1 import { Construction } from "lucide-react";
2
3 export default function Placeholder({ titulo, descricao }) {
4   return (
5     <div className="flex h-full flex-col">
6       <header className="border-b border-terra-line px-8 py-4">
7         <h1 className="font-display text-lg font-medium text-terra-navy">{titulo}</
h1>
8       </header>
9       <div className="flex flex-1 items-center justify-center px-8">
10         <div className="max-w-sm text-center">
11           <Construction className="mx-auto text-terra-ink-muted" size={22}
strokeWidth={1.5} />
12           <p className="mt-3 text-sm text-terra-ink-muted">{descricao}</p>
13           <p className="mt-1 text-sm text-terra-ink-muted">
14             Fase 2 do <span className="font-mono text-xs">PAINEL-STATUS.md</span>.
15             Usa o
16             Chat enquanto isso.
17           </p>
18         </div>
19       </div>
20     );
21 }
```

Recebe `titulo` e `descricao` como propriedades — é por isso que a mesma tela serve pras quatro páginas ainda não implementadas (Geral, Clima, Pragas, Operacional), cada uma só passando um texto diferente em `App.jsx` (seção 4.2). Quando uma dessas virar de verdade na Fase 2, o componente inteiro é substituído — não tem nada aqui pra reaproveitar além do texto que já foi escrito.

4.9 — index.css

ARQUIVO	PAPEL	DEPENDÊNCIA	CHAMADO POR
frontend/src/ index.css	os tokens de design — cor, tipografia — que todo o resto do frontend usa	Tailwind v4	todos os componentes, via classes utilitárias

```
1 @import url('https://fonts.googleapis.com/css2?
  family=Space+Grotesk:wght@500;600;700&family=IBM+Plex+Sans:wght@400;500;600&family=
  IBM+Plex+Mono:wght@400;500&display=swap');
2 @import "tailwindcss";
3
4 @theme {
5   /* Paleta — ancorada na marca real (mcp_relatorios/docs/marca-terra.md), não */
6   /* inventada solta. Verde é o único acréscimo: motivado pelo domínio (agro), */
7   /* usado só pra estado vivo/ativo, nunca decorativo. */
8   --color-terra-navy: #172944;
9   --color-terra-navy-hover: #3A4960;
10  --color-terra-navy-muted: #687485;
11  --color-terra-navy-tint: #DCDFE3;
12
13  --color-terra-canvas: #FAF9F6;
14  --color-terra-surface: #FFFFFF;
15  --color-terra-line: #E1E1E7;
16
17  --color-terra-ink: #1A1A1A;
18  --color-terra-ink-muted: #55606E;
19
20  --color-terra-sage: #3D7A5C;
21  --color-terra-sage-tint: #E8EFEB;
22  --color-terra-sage-dark: #2E5C45;
23
24  --color-terra-brick: #C0503D;
25  --color-terra-brick-tint: #FAECE8;
26
27  --font-display: "Space Grotesk", sans-serif;
28  --font-body: "IBM Plex Sans", sans-serif;
29  --font-mono: "IBM Plex Mono", monospace;
30 }
31
32 body {
33   background: var(--color-terra-canvas);
34   color: var(--color-terra-ink);
35   font-family: var(--font-body);
36 }
37
```

```
38 @layer components {
39   .campo {
40     @apply w-full rounded-md border border-terra-line bg-white px-3 py-2 text-sm
41       text-terra-ink placeholder:text-terra-ink-muted/60
42       focus:outline-none focus:ring-2 focus:ring-terra-navy/20 focus:border-
         terra-navy;
43   }
44 }
```

Tailwind v4 mudou o jeito de customizar tema — não existe mais um `tailwind.config.js` separado com um objeto `theme.extend`; agora as variáveis ficam direto no CSS, dentro de um bloco `@theme`. Cada variável `--color-terra-*` vira automaticamente uma classe utilitária: `--color-terra-navy` gera `bg-terra-navy`, `text-terra-navy`, `border-terra-navy`, etc., sem nenhum passo manual a mais.

A paleta não foi inventada solta — `#172944` é o azul extraído por amostragem de pixel do ícone oficial da marca (documentado em `mcp_relatorios/docs/marca-terra.md`), o mesmo usado nos relatórios em PDF. O verde (`terra-sage`) é o único acréscimo que não vem da marca original — motivado pelo domínio (agropecuário), usado só pra estado vivo/ativo (a etiqueta de fonte no chat), nunca como decoração solta.

ONDE MEXER

Quer mudar uma cor em todo o painel de uma vez? É aqui, uma linha só — por exemplo, trocar `--color-terra-navy` muda a cor principal em todo lugar que usa `bg-terra-navy` ou `text-terra-navy`, sem precisar caçar cada componente.

4.10 — vite.config.js

ARQUIVO	PAPEL	DEPENDÊNCIA	CHAMADO POR
frontend/ vite.config.js	configuração de build e do proxy de desenvolvimento	—	npm run dev, npm run build

```
1 import { defineConfig } from 'vite'
2 import react from '@vitejs/plugin-react'
3 import tailwindcss from '@tailwindcss/vite'
4
5 export default defineConfig({
6   plugins: [react(), tailwindcss()],
7   server: {
8     port: 5173,
9     proxy: {
10       '/api': { target: 'http://localhost:8001', changeOrigin: true },
11     },
12   },
13 })
```

Duas coisas: os plugins (React + Tailwind, os dois necessários pra esses frameworks funcionarem dentro do Vite), e o `proxy` — é a peça que faz `fetch('/api/...')` (seção 4.4) virar, por baixo dos panos, uma chamada pra `http://localhost:8001/api/...`. Isso só vale durante `npm run dev`; num build de produção (`npm run build`), esse proxy não existe mais, e o endereço real do backend precisaria ser configurado de outra forma (fora do escopo desta fase).

ONDE MEXER

Backend rodando em porta diferente de 8001? Troca o `target` aqui — é o único lugar do frontend inteiro que sabe esse endereço.

5. Fontes de dados, em profundidade

Esta seção junta e aprofunda algo que já apareceu espalhado nas seções 2 e 3: de onde vem cada dado que o painel usa, e os dois lugares diferentes onde essa informação precisa estar configurada.

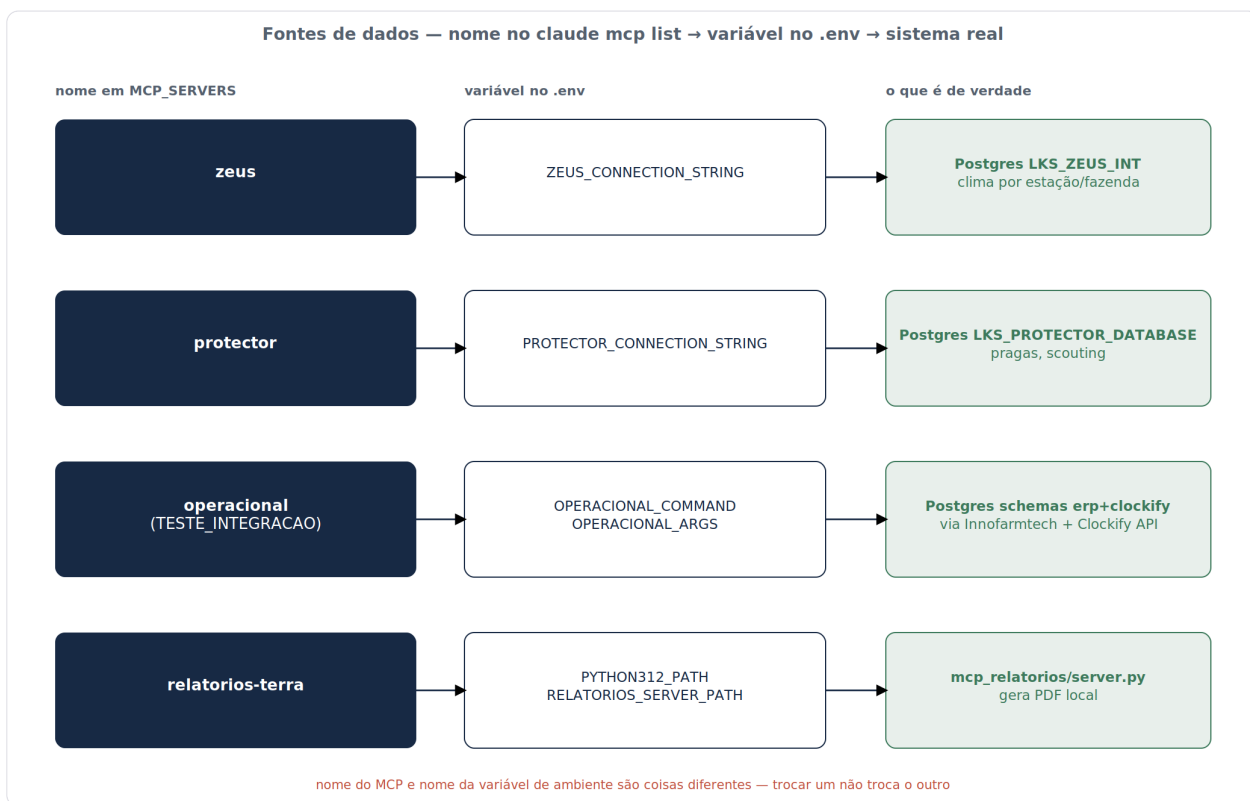


Figura 5 — nome interno, variável de ambiente, e sistema real, lado a lado.

Os quatro MCPs, um por um

Zeus — clima

Rede de estações meteorológicas em campo. Schema completo mapeado em `docs/mapeamento-lks-zeus-int.md`, no projeto raiz (fora deste painel) — pluviometria, temperatura, umidade, radiação solar, por estação/talhão/fazenda, mais previsão de curto/médio/longo prazo e status das estações. Banco: **Postgres**, conectado via `@modelcontextprotocol/server-postgres` (o pacote genérico de MCP pra Postgres — não é código escrito por vocês, é uma ferramenta de terceiro, mantida pela própria comunidade do protocolo MCP).

Protector — pragas

Monitoramento e scouting de pragas e doenças em campo — propriedades, talhões, safras, variedades, pontos de amostragem, leituras de indicadores de praga por ponto. Mesmo mecanismo de conexão do Zeus. **Pendência conhecida, não resolvida aqui:** a fórmula exata de "% de plantas infestadas" (média simples ou ponderada?) ainda depende de validação por agrônomo — isso bloqueia especificamente a página de Pragas na Fase 2, não bloqueia o chat (o agente pode consultar dado bruto sem aplicar a fórmula final).

Operacional — TESTE_INTEGRACAO

O mais diferente dos outros três. Não é uma base "só Postgres direto" — é um Postgres alimentado por um **pipeline Python próprio** (`python main.py`, em outra pasta do projeto, fora deste painel), com dois schemas:

- `erp` — 11 tabelas + 3 views, réplica do ERP interno da Terra (via Innofarmtech, `terra.innofarmtech.com.br/api/terra`): clientes, contratos, fazendas, produtos/serviços, colaboradores.
- `clockify` — 1 tabela (`time_entries`), horas apontadas por técnico/tarefa/tag, vindas da API do Clockify (`reports.api.clockify.me`), 365 dias corridos.

O nome `TESTE_INTEGRACAO` (no `claude mcp list`) é um nome de teste que ficou — funciona normalmente, mas vale renomear pra algo definitivo quando der (não é urgente). Diferente do Zeus/Protector, não há garantia de que o comando de conexão seja `npx @modelcontextprotocol/server-postgres` — pode ser um servidor MCP customizado escrito especificamente pra esse pipeline. `claude mcp get TESTE_INTEGRACAO` mostra o comando real.

relatorios-terra — geração de PDF

O único que não é fonte de *leitura* de dado — é uma *ação*: gera um PDF com a identidade visual da Terra a partir de conteúdo estruturado (texto, tabelas, gráficos de barra/linha). Roda localmente, como um script Python, sem banco de dado nenhum atrás. Documentado por completo em `mcp_relatorios/`, na raiz do projeto.

Os dois lugares onde uma base "existe"

Isto é o ponto mais importante desta seção inteira, porque já causou confusão antes nesta mesma conversa: **o nome que aparece no `claude mcp list` e a connection string do banco são duas informações diferentes, usadas em dois caminhos diferentes.**

Nome do MCP	Connection string
-------------	-------------------

Pra que serve	o agente (chat) saber chamar o servidor certo	consulta SQL direta, sem passar pelo agente (Fase 2)
Onde mora hoje	<code>MCP_SERVERS</code> , em <code>agente.py</code>	dentro dos <code>args</code> de cada entrada de <code>MCP_SERVERS</code> — mas só porque o zeus/protector passam ela pro <code>npx</code> como argumento
Como descobrir o valor real	<code>claude mcp list</code>	<code>claude mcp get <nome></code> — aparece dentro do comando completo

Ou seja: **trocar o nome de um MCP não troca pra onde ele conecta**, e vice-versa. Já a connection string, essa sim é reaproveitável — a mesma string que alimenta `ZEUS_CONNECTION_STRING` hoje (pro chat) vai ser exatamente a mesma que uma futura `consultas/zeus.py` (Fase 2) vai usar pra rodar SQL direto, só que sem passar pelo pacote `npx` — provavelmente com `psycpg` ou `sqlalchemy` direto em Python.

Isso significa que preencher o `.env` corretamente hoje já adianta trabalho da Fase 2 — a mesma variável `ZEUS_CONNECTION_STRING` vai ser reaproveitada, não substituída.

6. Configuração do MCP, em profundidade

MCP (Model Context Protocol) é um protocolo aberto — não é invenção da Terra nem da Anthropic sozinha — pra um programa de IA conversar com ferramentas externas de um jeito padronizado. Antes do MCP existir, cada integração (banco de dado, API, arquivo) precisava de código sob medida; com o MCP, qualquer servidor que fala o protocolo funciona com qualquer cliente que também fala — foi por isso que o Claude Code conseguiu usar o Zeus e o Protector sem vocês escreverem nenhum código de integração além do próprio servidor MCP. O painel faz a mesma coisa, só que com um cliente diferente (LangGraph, em vez do Claude Code).

Transporte stdio — o que significa na prática

Todos os quatro MCPs deste projeto usam `"transport": "stdio"`. Significa que a conversa entre o backend e cada MCP server acontece através da **entrada e saída padrão** de um processo — literalmente os mesmos canais que um programa de linha de comando usa pra imprimir texto na tela e ler o que você digita. O backend, em vez de "digitar", escreve mensagens do protocolo MCP formatadas em JSON; o servidor MCP, em vez de "imprimir na tela", escreve a resposta no mesmo formato.

Consequência prática: **cada MCP server é um processo separado do sistema operacional**, filho do processo do backend. Quando o backend sobe (seção 2.1), ele efetivamente executa algo equivalente a rodar `npx -y @modelcontextprotocol/server-postgres "postgresql://..."` na linha de comando, só que programaticamente, e mantém esse processo vivo enquanto o backend estiver de pé.

Isso é o mesmo motivo pelo qual, se você olhar o Gerenciador de Tarefas do Windows com o backend rodando, vai ver processos extras de `node` (pros MCPs via `npx`) e `python` (pro `relatorios-terra`) — um por MCP configurado. Encerrar o backend (`Ctrl+C` no terminal) deveria encerrar esses processos filhos junto; se sobrar algum "zumbi", é preciso encerrar na mão.

A ponte: langchain-mcp-adapters

MCP, sozinho, não sabe nada sobre LangGraph — é um protocolo genérico. Quem faz a tradução entre "ferramenta MCP" e "ferramenta que o LangGraph entende" é o pacote `langchain-mcp-adapters`, usado em `agente.py` através de duas chamadas:

- `MultiServerMCPClient(servidores)` — recebe o dicionário `MCP_SERVERS` (já filtrado por `_servidores_configurados()`) e sabe como subir cada processo e falar o protocolo com ele.
- `await client.get_tools()` — pergunta pra cada servidor "quais ferramentas você oferece?", recebe de volta o nome, a descrição e o formato dos parâmetros de cada uma, e traduz isso pro formato que o `create_react_agent` do LangGraph espera. É esse formato traduzido que o Claude efetivamente vê quando decide se vai chamar uma ferramenta.

Esse mecanismo foi **testado de ponta a ponta antes de virar código do painel** — não é suposição de que funcionaria. Subiu o `relatorios-terra` de verdade, listou as ferramentas, chamou uma delas diretamente (sem passar pelo modelo, só pra confirmar que o encanamento funciona) e conferiu que o PDF resultante existia no disco.

ONDE MEXER

Precisa saber exatamente o que um MCP oferece, sem abrir o código dele?

`claude mcp get <nome>` mostra o comando; rodar esse comando manualmente no terminal (fora do painel) e mandar uma mensagem MCP de `list_tools` mostra a lista bruta — mais avançado, raramente necessário no dia a dia.

7. Configuração do agente, em profundidade

O que "agente" quer dizer aqui

`create_react_agent`, do LangGraph, implementa um padrão chamado **ReAct** (raciocinar + agir, não confundir com a biblioteca React do frontend — nomes parecidos, coisas sem relação nenhuma). O modelo entra num laço: lê a pergunta e o histórico, decide se sabe responder direto ou se precisa de mais informação, e se precisar, escolhe uma ferramenta, chama ela, lê o resultado, e decide de novo — responde, ou chama outra ferramenta. Esse laço se repete até o modelo decidir que já tem o suficiente pra responder em texto final.

Nada disso é escrito à mão em `agente.py` — é o comportamento padrão do `create_react_agent`, configurado só com o modelo e a lista de ferramentas:

```
1 modelo = ChatAnthropic(model=MODELO)
2 agente = create_react_agent(modelo, tools)
```

Model tiering — por que claude-sonnet-5 por padrão

`MODELO = os.environ.get("PAINEL_MODELO", "claude-sonnet-5")` segue uma decisão de arquitetura que já vem do projeto original (não é nova deste painel): **modelo forte por padrão em todo nó do sistema** — trocar por um modelo mais barato só depois de um teste comparativo confirmando que a qualidade não cai, nunca por suposição. Hoje o agente inteiro é "um nó só" (o laço ReAct), então essa variável controla o sistema inteiro; quando a Fase 3 do `PROJETO-STATUS.md` original (montar o LangGraph "de verdade", com nós separados por etapa) acontecer, é esperado que cada nó tenha sua própria variável de modelo, não uma só.

astream_events — o que é a versão "v2"

`astream_events` existe em duas versões de formato de evento por razões históricas da própria LangChain; `version="v2"` é a atual e estável. Além dos dois tipos de evento que `perguntar_streaming()` usa (`on_tool_start` e `on_chat_model_stream`), existem vários outros que este código ignora de propósito — `on_tool_end` (ferramenta terminou), `on_chain_start` / `on_chain_end` (início/fim de cada etapa interna do grafo), entre outros. Foram deixados de fora porque nenhum deles tem uma representação útil pra mostrar no chat hoje — mas estão disponíveis se uma futura necessidade aparecer (por exemplo,

mostrar quanto tempo uma ferramenta demorou, usando o intervalo entre `on_tool_start` e `on_tool_end`).

O histórico da conversa

Cada chamada de `perguntar_streaming()` recebe o `historico` inteiro (todas as mensagens anteriores da conversa, vindas do frontend) e manda ele de volta pro modelo junto com a pergunta nova — o backend não guarda memória de conversa nenhuma entre chamadas, quem guarda é o React (`Chat.jsx` , seção 4.7), em memória do navegador. Fechar a aba apaga a conversa; não existe hoje persistência de histórico em banco de dado. Isso também significa que **não há limite de tamanho de conversa aplicado** — uma conversa muito longa eventualmente esbarraria no limite de contexto do modelo, mas isso não foi testado nem tratado nesta fase.

ONDE MEXER

Quer guardar histórico de conversa entre sessões (sobreviver a fechar a aba)? Isso não existe ainda — precisaria de uma tabela nova (Postgres, provavelmente no mesmo banco operacional) e mudanças tanto no backend quanto no `Chat.jsx` . Fora do escopo desta fase.

8. Autenticação, em profundidade

Três mecanismos trabalhando juntos: **bcrypt** (proteger a senha guardada), **JWT** (provar quem é você depois de logado, sem logar de novo a cada clique), e **cookie httpOnly** (guardar essa prova no navegador com segurança). Cada um resolve um problema diferente — vale entender os três separadamente antes de ver como se encaixam.

bcrypt — por que não comparar senha com senha

`usuarios.yaml` nunca guarda a senha em si — guarda um **hash**: o resultado de passar a senha por uma função matemática que não tem volta (não existe "deshash", só recalculando o hash de um palpite e comparar). `bcrypt` especificamente adiciona um **salt** — um valor aleatório único por senha, embutido no próprio hash — que garante que duas pessoas com a mesma senha tenham hashes completamente diferentes guardados. Sem isso, quem tivesse acesso ao arquivo poderia comparar hashes iguais entre usuários diferentes e descobrir que compartilham senha, mesmo sem saber qual é.

Login funciona assim: `bcrypt.checkpw(senha_digitada, hash_guardado)` recalcula o hash da senha digitada usando o *mesmo salt* que já está guardado no hash original, e compara os dois resultados. Bate = senha certa. Em nenhum momento a senha digitada é comparada, como texto, com nada guardado.

JWT — o que tem dentro de um token

Um JWT (JSON Web Token) tem três partes, separadas por pontos: `cabecalho.corpo.assinatura`. As duas primeiras são só texto codificado em base64 — **qualquer pessoa consegue decodificar e ler**, não é criptografia, é só uma forma compacta de representar dado. O token que este sistema gera, decodificado, tem essa cara:

```
1 {
2   "sub": "admin",
3   "nome": "Administrador",
4   "exp": 1755043200
5 }
```

(`sub` é o usuário, `exp` é quando expira, em formato Unix timestamp — `SESSAO_DIAS`, em `auth.py`, controla esse número.)

O que impede alguém de simplesmente **escrever** esse JSON à mão e se passar por `admin` é a terceira parte, a **assinatura** — calculada usando `JWT_SECRET` (do `.env`) como

chave. `jwt.decode()`, em `usuario_atual()`, recalcula essa assinatura a partir do cabeçalho e corpo recebidos e confere se bate com a que veio no token. Sem saber o `JWT_SECRET`, é possível ler um token existente mas não *forjar* um novo que passe na validação.

Isso é exatamente por que `JWT_SECRET` precisa ser um valor aleatório, único, guardado só no `.env` (nunca commitado) — quem tiver esse valor consegue criar uma sessão válida pra qualquer usuário, sem saber a senha de ninguém. É o segredo mais sensível deste projeto inteiro.

Cookie `httpOnly` — onde o token fica guardado

Depois de validado, o JWT precisa viajar de volta pro navegador e ser guardado em algum lugar que sobreviva entre uma página e outra. A escolha aqui foi **cookie `httpOnly`**, não `localStorage` (uma alternativa comum). A diferença que importa: um cookie `httpOnly` é **inacessível a JavaScript** — nem o próprio código do frontend consegue ler o valor dele, só o navegador, que manda ele automaticamente em toda requisição pro mesmo domínio. Isso protege contra um tipo específico de ataque (XSS — injeção de script malicioso na página) que conseguiria roubar um token guardado em `localStorage`, mas não consegue nem enxergar um cookie `httpOnly`.

```
1 response.set_cookie(  
2     "painel_sessao", token,  
3     httpOnly=True, samesite="lax", max_age=7 * 24 * 3600,  
4 )
```

`samesite="lax"` é outra camada — impede que o cookie seja mandado em requisições vindas de *outros sites* tentando chamar a API do painel escondido num link ou formulário malicioso.

O que "simples" quer dizer aqui, com honestidade

Este mecanismo é apropriado pra uma equipe pequena, interna, numa rede confiável — que foi exatamente o pedido original ("algo simples que eu posso controlar agora, depois evoluímos"). Vale ser direto sobre o que **não** existe ainda, caso o uso cresça:

- Sem limite de tentativas de login (força bruta de senha não é bloqueada)
- Sem revogação de sessão — uma vez emitido, um token é válido até expirar, mesmo que o usuário seja removido de `usuarios.yaml` depois
- Um único papel — não existe "admin" vs "usuário comum", todo mundo que loga tem o mesmo acesso

- Sem log de quem acessou o quê

Nenhum desses é difícil de adicionar quando fizer sentido — só não foi construído agora porque não foi pedido, e adicionar sem necessidade real custaria tempo sem benefício imediato.

9. Onde mexer — índice rápido

Todo "Onde mexer" já espalhado pelo documento, num lugar só, pra consulta rápida sem precisar reler as seções inteiras.

Eu quero...	Edito
Trocar a senha de um usuário	<code>python criar_usuario.py</code> (nunca o YAML na mão) — §3.4
Adicionar um usuário novo	<code>python criar_usuario.py</code> — §3.4
Trocar quanto tempo a sessão dura	<code>SESSAO_DIAS</code> em <code>backend/auth.py</code> — §3.2
Adicionar um MCP server novo (5º)	<code>MCP_SERVERS</code> em <code>backend/agente.py</code> , mais o <code>.env</code> — §3.3
Trocar o modelo Claude usado	<code>PAINEL_MODELO</code> no <code>.env</code> do backend — §3.5, §7
Publicar o frontend em outro endereço	<code>ORIGEM_FRONTEND</code> no <code>.env</code> do backend — §3.1
Implementar a página Geral/Clima/Pragas/Operacional (Fase 2)	<code>frontend/src/App.jsx</code> (trocar o Placeholder) + criar o componente novo — §4.2, §4.8
Mudar o texto de boas-vindas do chat vazio	<code>EstadoVazio</code> , em <code>frontend/src/pages/Chat.jsx</code> — §4.7
Mudar como o nome de um MCP aparece na etiqueta do chat	<code>NOMES_LEGIVEIS</code> , em <code>frontend/src/pages/Chat.jsx</code> — §4.7
Adicionar item no menu lateral	<code>ITENS</code> , em <code>frontend/src/components/Layout.jsx</code> — §4.5
Mudar uma cor da marca em todo o painel	<code>frontend/src/index.css</code> , bloco <code>@theme</code> — §4.9
Backend rodando em porta diferente de 8001	<code>target</code> em <code>frontend/vite.config.js</code> — §4.10
Descobrir a connection string real de um MCP	<code>claude mcp get <nome></code> , no terminal — §5

10. Erros conhecidos e como diagnosticar

Dois bugs reais apareceram construindo este painel — não são hipóteses, foram encontrados rodando o sistema de verdade, e já estão corrigidos no código entregue. Documentados aqui porque o *tipo* de erro pode reaparecer de outras formas conforme o projeto cresce.

Já corrigidos, mas vale entender

Sintoma	Causa real	Onde foi corrigido
Erro <code>password cannot be longer than 72 bytes</code> ao gerar hash de senha	<code>passlib</code> incompatível com versões atuais do pacote <code>bcrypt</code> (bug conhecido da lib, não do projeto)	<code>auth.py</code> usa <code>bcrypt</code> direto, não <code>passlib</code> — §3.2
Chat trava, navegador não recebe nenhuma resposta, sem mensagem de erro	Exceção no meio do <code>async for</code> de streaming derrubava a conexão em silêncio	<code>try/except</code> em volta do laço inteiro, em <code>perguntar_streaming()</code> — §3.3

Diagnóstico — por sintoma

"Failed to fetch" ou erro de rede no navegador, em qualquer ação

1. O backend está rodando? Confere o terminal onde rodou `uvicorn` — sem erro visível ali, sem processo, sem funcionar.
2. Backend na porta 8001 mesmo? Se mudou a porta, o `vite.config.js` (§4.10) precisa apontar pro lugar certo.

Login sempre dá "Usuário ou senha incorretos", mesmo com a senha certa

1. `usuarios.yaml` existe de verdade dentro de `backend/` (não só o `.example`)?
2. O login digitado bate exatamente com a chave no YAML (sensível a maiúscula/minúscula)?
3. Se criou o hash na mão em vez de usar `criar_usuario.py` — essa é quase sempre a causa. Refaz pelo script.

Login funciona, mas toda rota depois disso devolve 401

1. `JWT_SECRET` está preenchido no `.env` ? Sem ele, `usuario_atual()` recusa tudo, de propósito (§3.2, §8).
2. O navegador está de fato mandando o cookie? Ferramentas de desenvolvedor do navegador → aba Network → conferir se a chamada tem `Cookie: painel_sessao=...` nos cabeçalhos.

Chat responde "Nenhum MCP server configurado ainda"

Nenhuma das quatro entradas de `MCP_SERVERS` passou no filtro de `_servidores_configurados()` (§3.3) — todas as variáveis de ambiente relevantes estão vazias no `.env`. Confere se o arquivo se chama exatamente `.env` (não `.env.txt` nem `env`) e está dentro de `backend/`, não na raiz do projeto.

Uma pergunta específica não chama a ferramenta esperada

Isso não é um bug de código — é o modelo decidindo, em cada pergunta, se e qual ferramenta chamar (§7). Perguntas ambíguas podem levar a decisões inesperadas. A etiqueta colorida acima de cada resposta (§4.7) sempre mostra qual ferramenta foi de fato usada — é a forma mais rápida de confirmar se o agente interpretou a pergunta do jeito esperado.

Onde olhar quando nada disso resolve

O terminal onde o `uvicorn` está rodando mostra o traceback completo de qualquer erro do backend — é sempre o primeiro lugar a olhar. Erros do frontend aparecem no console do navegador (F12 → aba Console).